Name: Keerthi

Program: Azure Data Engineering

PROJECT 3: Data Warehousing with Azure Synapse Analytics

1. INTRODUCTION:

In this project, we utilize Azure Synapse Analytics to design and implement a scalable data warehouse solution for a financial institution.

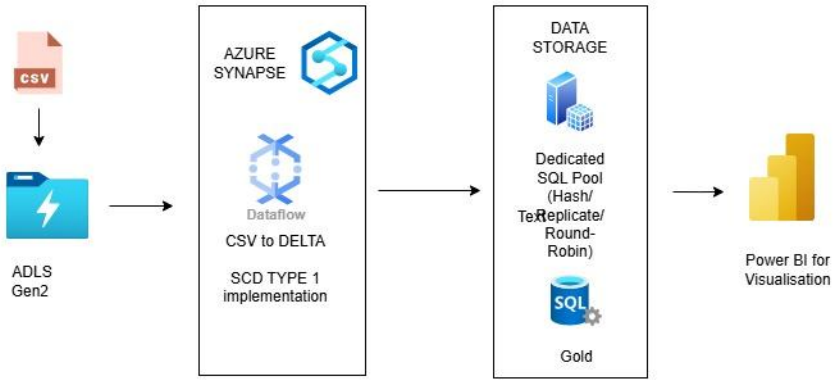
Azure Synapse Analytics integrates big data and data warehousing capabilities, allowing seamless ingestion, transformation, and analysis of data using both serverless and dedicated SQL pools.

Objective

The objective of this project is to build an end-to-end data warehousing pipeline that:

- Ingests raw transaction data
- Transforms and stores data efficiently
- Implements Slowly Changing Dimensions (SCD Type 1)
- Optimizes storage using distribution strategies

2. ARCHITECTURE DIAGRAM:



ARCHITECTURE DIAGRAM

3. Data Ingestion and Processing

1. Data Ingestion

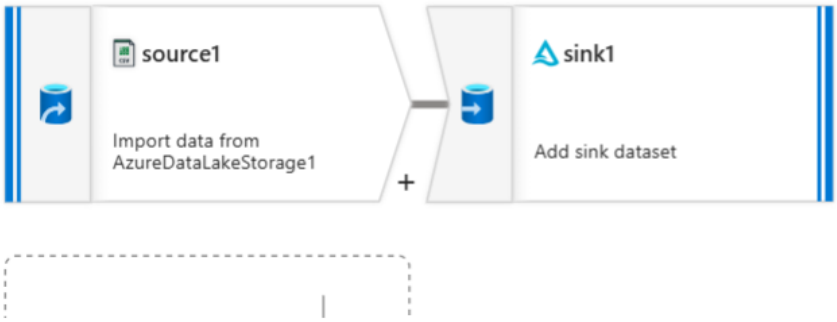
Data is ingested from CSV files into ADLS Gen2.

2. Dataflows

Dataflows are used to:

- Clean and transform data
- Convert formats (CSV to Delta)
- Implement SCD Type 1 logic

CSV to DELTA:



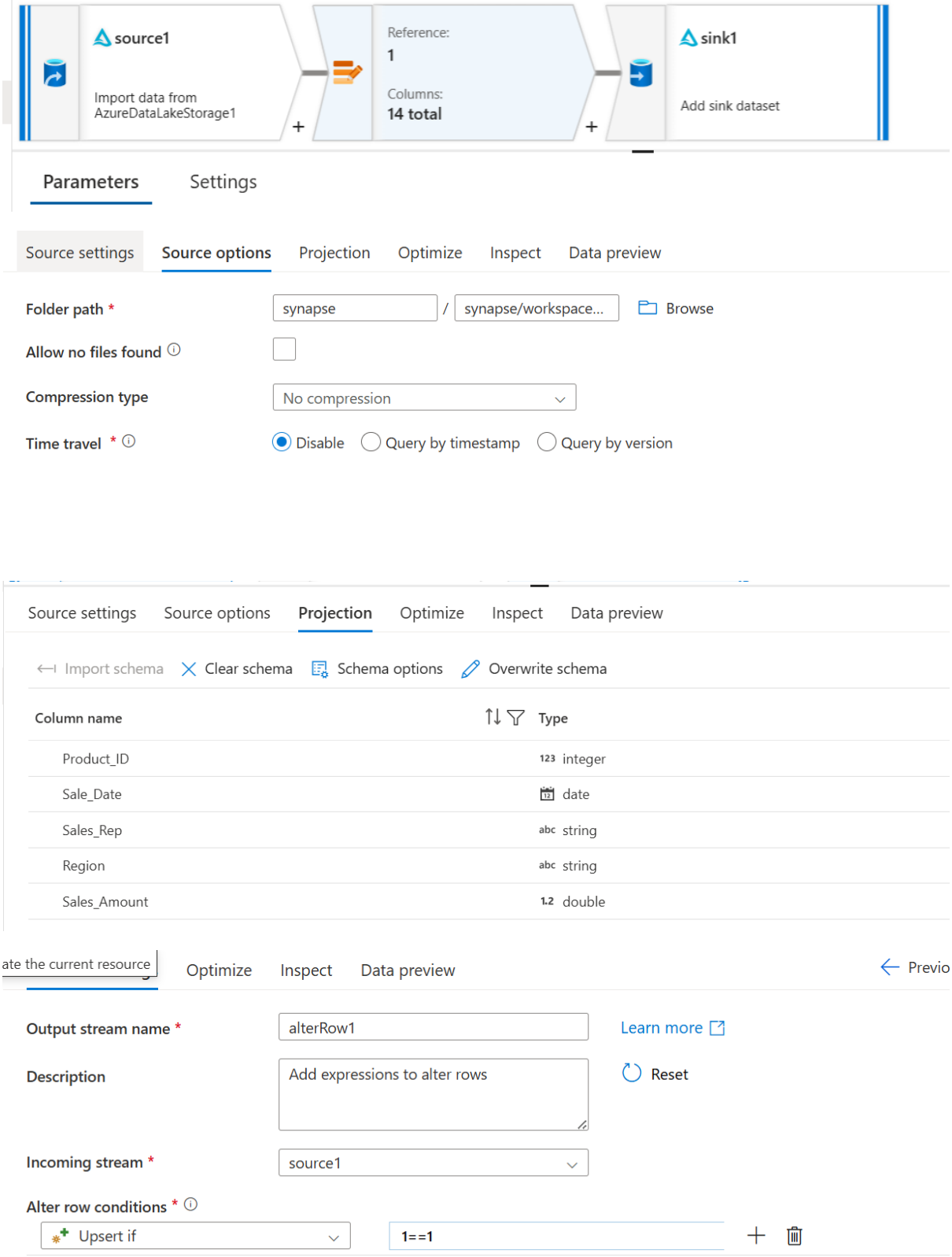
Slowly Changing Dimensions (SCD) Type 1 Implementation:

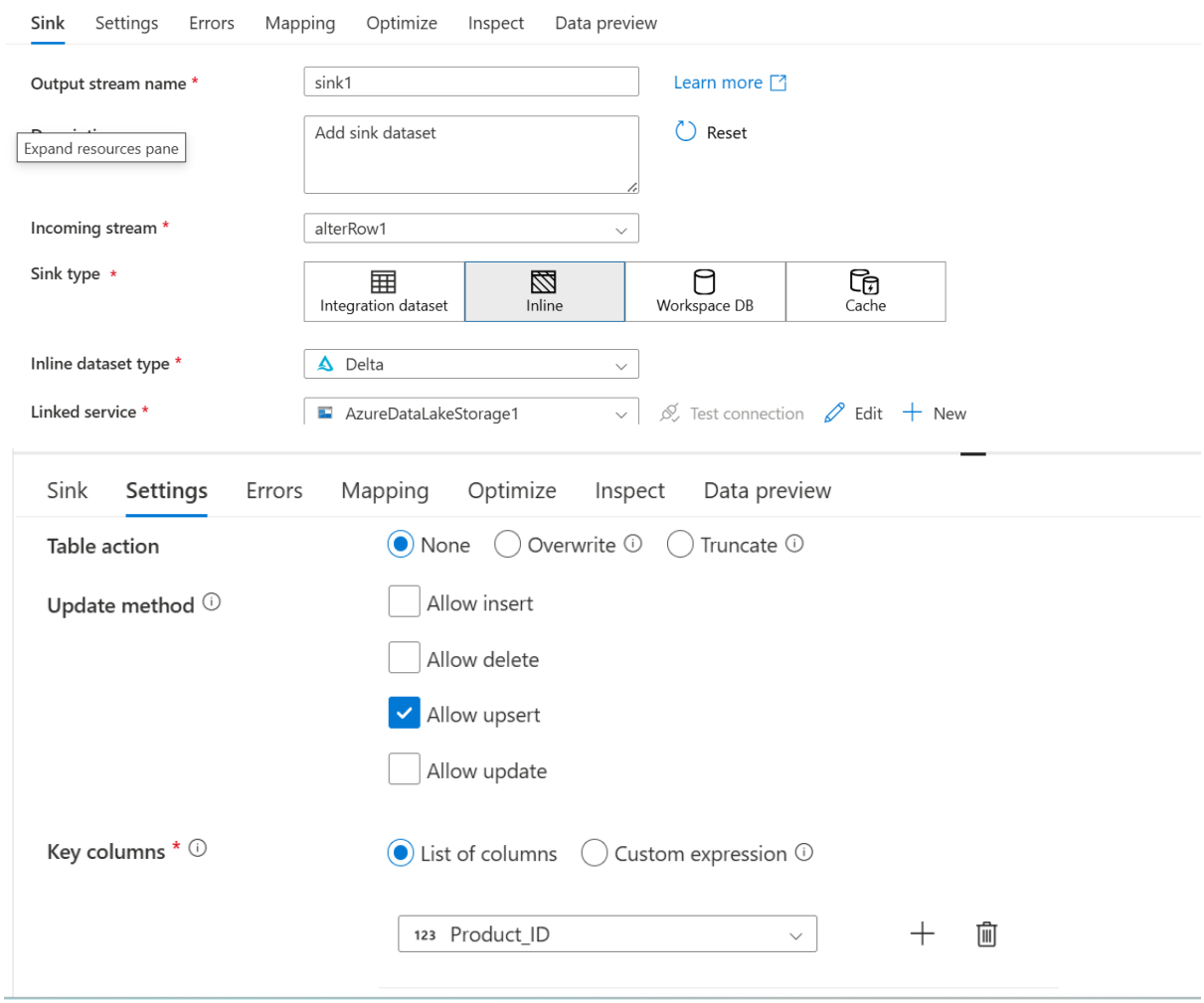
SCD handle changes in dimension data over time.

SCD Type 1:

- Overwrites old data with new data
- No historical tracking
- Simple and efficient

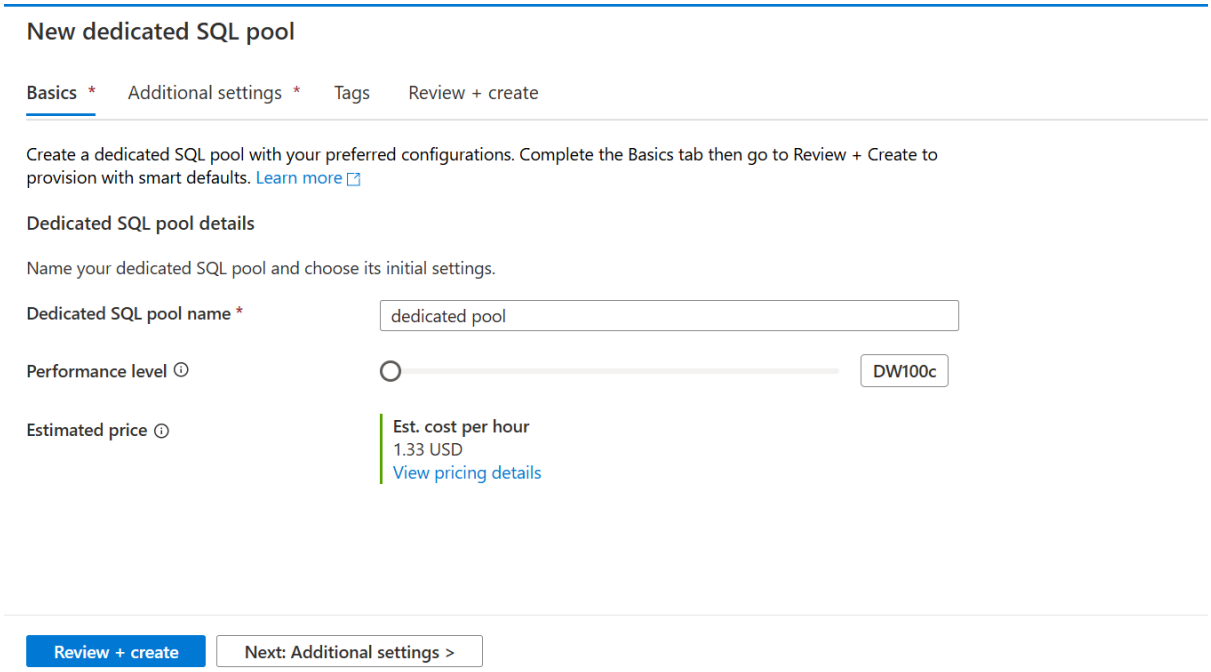
Used when historical data is not required.


source1
Import data from AzureDataLakeStorage1
Reference:
1
Columns:
14 total
sink1
Add sink dataset
Parameters
Settings
Source settings
Source options
Projection
Optimize
Inspect
Data preview
Folder path *
synapse
synapse/workspace...
Browse
Allow no files found
Compression type
No compression
Time travel *
Disable
Query by timestamp
Query by version
Source settings
Source options
Projection
Optimize
Inspect
Data preview
Import schema
Clear schema
Schema options
Overwrite schema
Column name
Type
Product_ID
integer
Sale_Date
date
Sales_Rep
string
Region
string
Sales_Amount
double
ate the current resource
Optimize
Inspect
Data preview
Previo
Output stream name *
alterRow1
Learn more
Description
Add expressions to alter rows
Reset
Incoming stream *
source1
Alter row conditions *
Upsert if
1==1



CREATING THE TABLES USING DEDICATED SQL POOL:

A dedicated SQL pool provides allocated compute resources for large-scale data processing.



Distribution Types:

a. **Hash Distribution**

- Distributes rows based on a hash of a column
- Best for large tables and joins

b. **Round-Robin Distribution**

- Evenly distributes data across all nodes
- Good for staging tables

c. **Replicated Tables**

- Copies entire table to all nodes
- Ideal for small dimension tables

CREATING TABLE USING HASH DISTRIBUTION:

```sql
CREATE TABLE FactSales
(
    Product_ID INT NOT NULL,
    Sale_Date DATE,
    Sales_Amount FLOAT,

    CONSTRAINT PK_FactSales
        PRIMARY KEY NONCLUSTERED (Product_ID) NOT ENFORCED
)
WITH
(
    DISTRIBUTION = HASH(Product_ID),
    CLUSTERED COLUMNSTORE INDEX
);
```

```sql
CREATE TABLE FactSalesDetailed
(
    Product_ID INT NOT NULL,
    Region_and_Sales_Rep VARCHAR(200) NOT NULL,
    Sale_Date DATE,
    Sales_Amount FLOAT,
    Quantity_Sold INT,
    Discount FLOAT,
    Payment_Method VARCHAR(50),
    Sales_Channel VARCHAR(50),
    Customer_Type VARCHAR(50),

    CONSTRAINT PK_FactSalesDetailed
        PRIMARY KEY NONCLUSTERED (Product_ID, Sale_Date) NOT ENFORCED
)
WITH
(
    DISTRIBUTION = HASH(Product_ID),
    CLUSTERED COLUMNSTORE INDEX
```

CREATING TABLE USING ROUND-ROBIN:

```sql

--Round Robin

CREATE TABLE DimSalesRep
(
    Region_and_Sales_Rep VARCHAR(200) NOT NULL,
    Sales_Rep VARCHAR(100),
    Region VARCHAR(100),

    CONSTRAINT PK_DimSalesRep
        PRIMARY KEY NONCLUSTERED (Region_and_Sales_Rep) NOT ENFORCED
)
WITH
(
    DISTRIBUTION = ROUND_ROBIN,
    CLUSTERED COLUMNSTORE INDEX
);
```

CREATING TABLE USING REPLICATE:

```sql
--Replicate

CREATE TABLE DimProduct
(
    Product_ID INT NOT NULL,
    Product_Category VARCHAR(100),
    Unit_Cost FLOAT,
    Unit_Price FLOAT,

    CONSTRAINT PK_DimProduct
        PRIMARY KEY NONCLUSTERED (Product_ID) NOT ENFORCED
)
WITH
(
    DISTRIBUTION = REPLICATE,
    CLUSTERED COLUMNSTORE INDEX
);
```

INSERTING DATA INTO THE CREATED TABLES (works only when the files are PARQUET format):

```sql
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'NCPLncpl@123123!';

--Now Create Credential using Storage Account Key:
CREATE DATABASE SCOPED CREDENTIAL MyStorageCredentials
WITH
    IDENTITY = 'Storage Account Key',
    SECRET = 'wFmL19BxnDC9wOeuiuwwriP9bhsDAeMrbKTzns5sFeDsdQ9oNhOOdi20W2uDyw5eydlLxfqxZH83+AStC+ro9g==';


--inserting data into the created tables

COPY INTO dbo.FactSales
FROM 'https://adlskg.dfs.core.windows.net/synapse/workspaces/synapsekg/gold_parquet/'
WITH (
    FILE_TYPE = 'PARQUET',
    CREDENTIAL = (IDENTITY = 'Storage Account Key', SECRET = 'wFmL19BxnDC9wOeuiuwwriP9bhsDAeMrbKTzns5sFeDsdQ9oNhO
);
```

```sql
-- This is auto-generated code
SELECT
    TOP 100 *
FROM
    OPENROWSET(
        BULK 'https://adlskg.dfs.core.windows.net/synapse/synapse/workspaces/synapsekg/gold1/',
        FORMAT = 'DELTA'
    ) AS [result]

```

QUERYING GOLD TABLE:

```sql
SELECT *
FROM OPENROWSET(
    BULK 'https://adlskg.dfs.core.windows.net/synapse/synapse/workspaces/synapsekg/gold',
    FORMAT = 'DELTA'
) AS rows;


CREATE DATABASE scd_db;
USE scd_db;

CREATE VIEW scd_view1 AS
SELECT *
FROM OPENROWSET(
    BULK 'https://adlskg.dfs.core.windows.net/synapse/
synapse/workspaces/synapsekg/gold',
    FORMAT = 'DELTA'
) AS rows;
```

CREATING VIEWS USING SERVERLESS SQL POOL TO QUERY THE DATA:

```sql
SELECT *
FROM OPENROWSET(
    BULK 'https://adlskg.dfs.core.windows.net/synapse/synapse/workspaces/synapsekg/gold',
    FORMAT = 'DELTA'
) AS rows;


CREATE DATABASE scd_db;
USE scd_db;

CREATE VIEW scd_view1 AS
SELECT *
FROM OPENROWSET(
    BULK 'https://adlskg.dfs.core.windows.net/synapse/
synapse/workspaces/synapsekg/gold',
    FORMAT = 'DELTA'
) AS rows;
```

4. CONCLUSION:

This project demonstrates the implementation of a modern data warehouse using Azure Synapse Analytics by integrating data ingestion, transformation, and reporting to build a scalable and efficient solution for analyzing customer transactions. Through this project, key concepts such as data warehousing fundamentals were explored, along with the implementation of Slowly Changing Dimensions (SCD Type 1) and the use of different distribution strategies to optimize performance and storage.